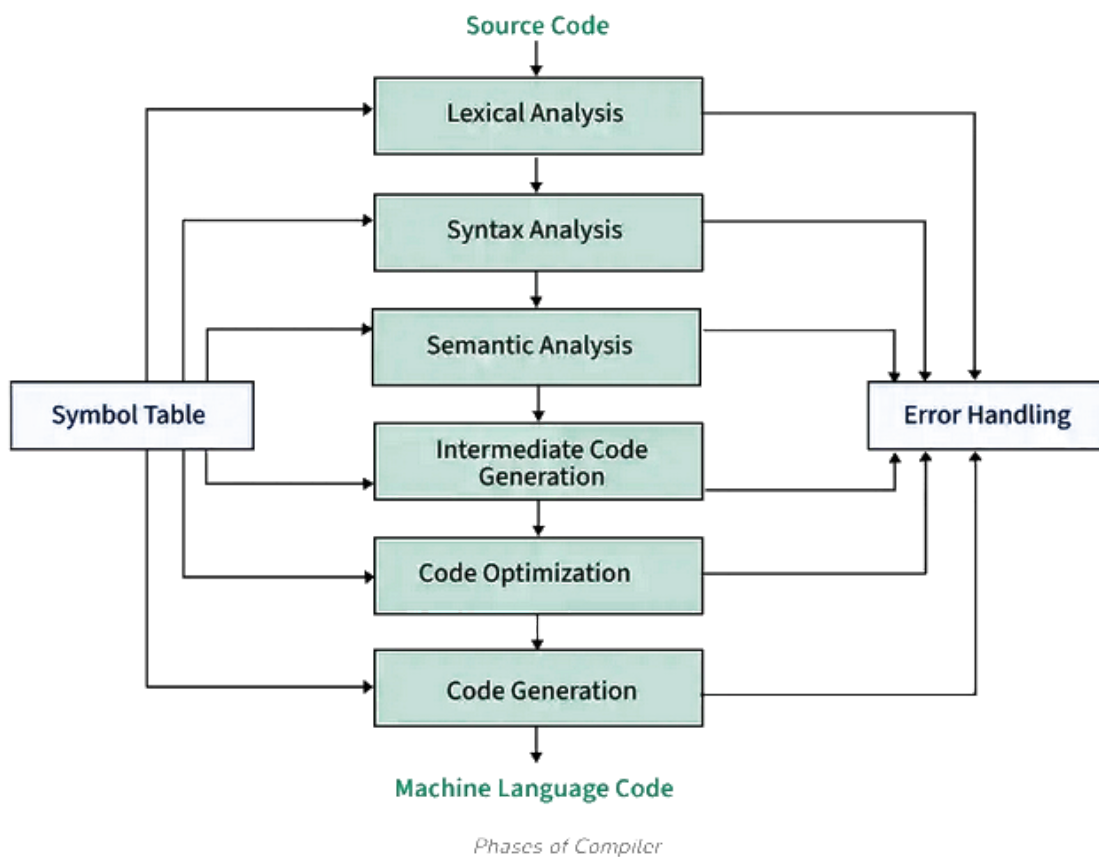


Application of DFAs in Compiler Construction

A compiler is a program that translates code written in a higher level programming language into a lower level language such as machine language that a computer can execute. Most compilers have 6 phases which are lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization and code generation.



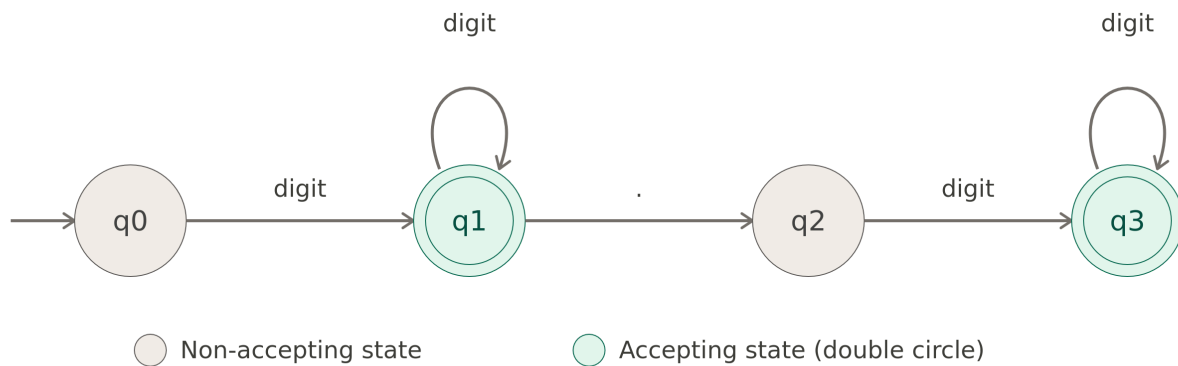
Recognizing words using DFAs

Lexical analysis is the first phase of a compiler which converts the source code into tokens. A token is the smallest meaningful unit of grammar e.g keyword or symbol. Lexical analyzers do this by reading the source input character by character and

then comparing them to a list of known symbols to determine which kind of tokens they are. This process can be simplified through the use of DFAs.

In a DFA, you have a set of *states* with *transitions* between them, forming a graph. At any point in time, the machine is “in” exactly one state. It gets to other states by following transitions. When you use a DFA for lexical analysis, each transition is a character that gets matched from the string. Each state represents a set of allowed characters.

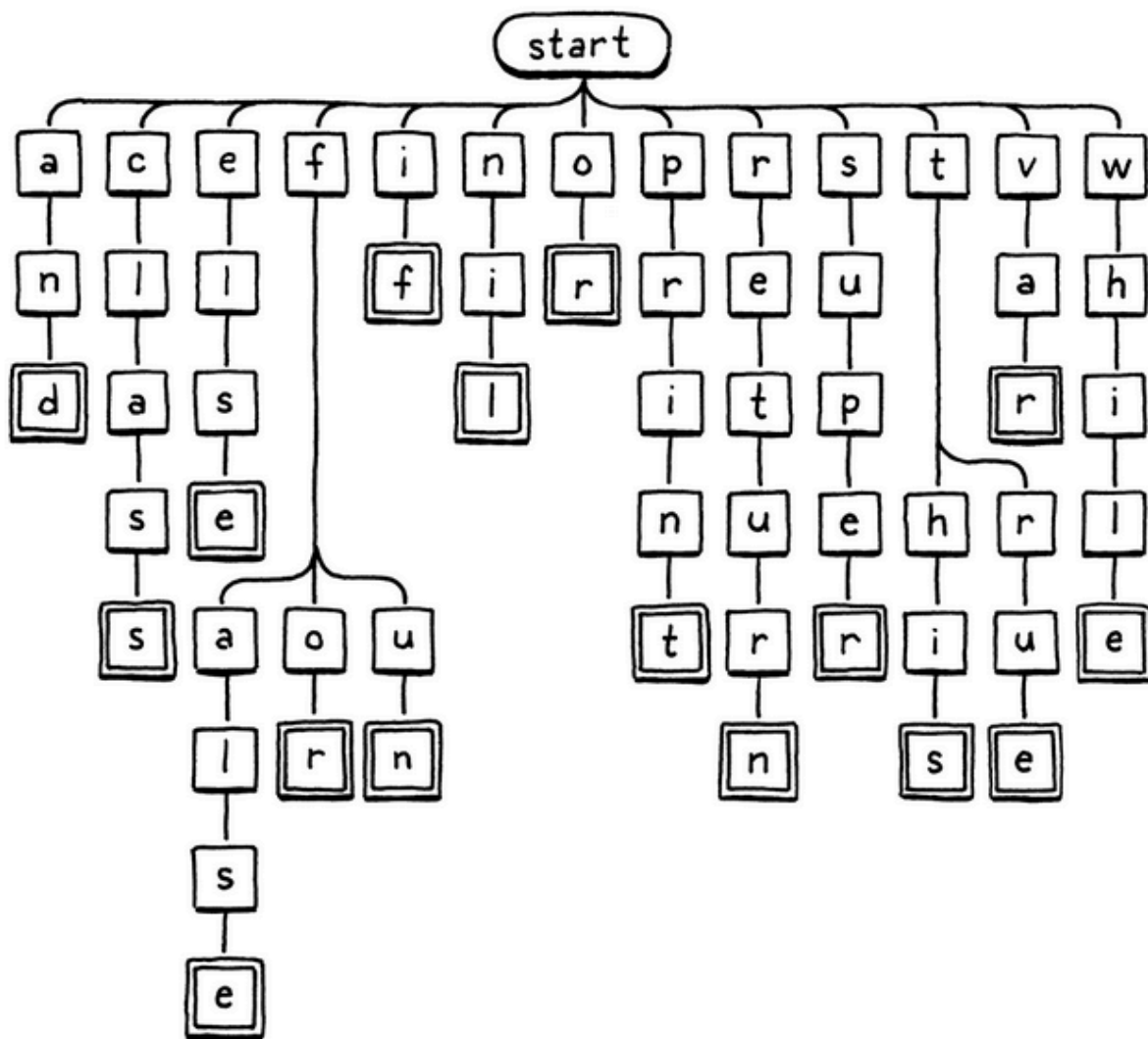
For example, suppose we want to recognize number literals such as `42` or `3.14`. We can build a DFA where each state captures how much of a valid number has been read so far. The machine starts in state `q0`. Reading a digit moves it to `q1`, which means "at least one digit has been seen", this state is accepting, since a string like `42` is already a complete, valid integer on its own. From `q1`, another digit loops back to `q1` (so integers of any length are allowed), while a decimal point moves the machine to `q2`. State `q2` is not accepting, because a number can't legally end right after a decimal point, `3.` isn't a valid literal. From `q2`, a digit moves to `q3`, an accepting state representing a complete decimal number like `3.1`, and further digits loop back to `q3` to allow more decimal places. Any input that doesn't follow one of these paths, such as two consecutive decimal points, has nowhere to go and is rejected.



The same process can be repeated for recognizing keywords such as `if`, `while`, or `return`. Since a keyword is a fixed, literal sequence of characters, its DFA is actually simpler than the one for number literals: each state expects exactly one specific character and moves to the next state in the chain when it sees it. Only after the last character of the word has been matched does the machine land in an accepting state. Recognizing `if`, for example, takes just three states: the start state moves to a second state on reading `i`, and that second state moves to an accepting third state on reading `f`. Any other character at any point sends the machine to a non-accepting state, since nothing else matches that keyword.

In practice, a lexer doesn't build one isolated DFA per keyword and run them all separately. Instead, the DFAs for every keyword get merged into a single combined machine, with shared prefixes folding into shared states. `if` and `int`, for instance, would share their first transition on `i` before diverging on `f` versus `n`. This is the same idea as a trie, just

expressed as a DFA.



There's also a shortcut many real compilers use instead: because the character pattern for identifiers (a letter followed by more letters or digits) already covers every keyword as a special case, the lexer first matches the input as a generic identifier, then checks the resulting string against a lookup table of reserved words. If it finds a match, the token gets reclassified as a keyword rather than an identifier. This sidesteps the need to encode every keyword as its own chain of states at all.

Building a dedicated DFA by hand for every kind of token, whether for number literals, keywords, identifiers, or operators,

illustrates the underlying idea well, but real lexical analyzers don't construct these by hand. Tools like `lex` and `flex` take a set of regular expressions describing each token type and automatically generate a single combined DFA that recognizes all of them in one pass over the source code, character by character. Once a token has been identified this way, it gets handed off to the next phase of the compiler, syntax analysis, which checks whether the resulting sequence of tokens actually follows the grammar of the language. DFAs, in other words, aren't just a theoretical convenience; they're the practical mechanism that makes the very first phase of compilation fast, predictable, and easy to reason about.

References

Cooper, K. D., & Torczon, L. (2022). *Engineering a Compiler* (3rd ed.). Morgan Kaufmann.

Nystrom, R. (2021). *Crafting Interpreters*. Genever Benning.

Sandler, N. (2024). *Writing a C Compiler: Build a Real Programming Language from Scratch*. No Starch Press.